

MANGAL データ索引 設計仕様 ―― 項目定義(レビュー用ドラフト)

作成: 2026-06-15 / 目的:
索引を整備する前に「何を・どこから・どう入れるか」を確定する。実装はこの承認後。本書はレビュー用。

0. 一行サマリ

公開データ(data/manga.v2)を真とし、SQLiteマスター索引(全項目・全文検索・被覆集計)と軽量サイト検索索引(ブラウザ即時検索用の最小JSON)の二層を、promote毎に再生成する。これで「検索が毎回遅い」「何が入って何が欠けているか即答できない」を根治する。

1. 背景と原則

- 問題: 66,582本のYAML(data/manga.v2)を毎回走査して検索・監査している→遅い。今日のISBN/表紙調査も数分の走査が必要だった。
- 既存資産: ① .cache/v2-index.json(69,506作・work級・項目少・JSON線形=キー検索/全文検索に非対応)。② .cache/db-v2.sqlite(種2の生派生層=166,441 series・公開前で別物。表紙/あらすじ等は0=promote時付与のため)。→どちらも『公開データの最新・全項目・検索可能な1枚』ではない。
- 原則1 真の源:
公開セット=data/manga.v2(promote後の出力)。db-v2(生層)ではない=実際に表示する姿と一致させる。
- 原則2 索引はキャッシュ:
manga.v2からいつでも再生成可。重い実体はgit追跡しない(レシピ=スクリプトだけ追跡)。
- 原則3 二層分離: 開発/監査用(全部入りSQLite)と、サイト配信用(極小JSON)は要件が逆(網羅 vs 軽さ)なので分ける。
- 原則4 再導出可能なものは焼かない:
価格/アフィリエイト等は楽天キャッシュからjoinできるので索引には“最小限/フラグ”で持つ。

2. 成果物(2種)

成果物	形式 / 置き場	役割	サイズ目安
A. マスター索引	SQLite .cache/mangal-index.sqlite	開発・監査・収穫対象抽出・将来のWorker/API バックエンド。全項目+全文検索(FTS5)+被覆集計。	~150-250MB
B. 軽量サイト索引	gzip JSON out/search-index/*.json.gz	ブラウザ即時検索・トップ/一覧の初期表示。最小項目のみ 。かな頭文字でシャード化し遅延ロード。(現状トップが全DBを送る問題の解決も兼ねる)	~2-3MB(gz)

A は『正確さ・網羅』、B は『軽さ・初動』。B は A から機械生成する派生物。

3. テーブル構成(粒度)

テーブル	粒度	主な用途
works	1作(=1公開ページ)1行	一覧・検索・作品単位の被覆・ソート
volumes	1巻1行	ISBN/表紙/発売日/価格の被覆・ストアリンク・収穫対象抽出
authors / work_authors	著者1行 / 作品×著者	著者検索・50音索引・著者ページ・原作/作画分離
works_fts (FTS5)	works対応の全文索引	題・かな・ローマ字・別名・著者名の即時部分一致検索
coverage (ビュー)	集計1行	『何が入って何が欠けているか』を1クエリで

4. 項目定義 ―― これが本体

型は SQLite 表記。源『v2』=data/manga.v2、『算出』=ビルド時計算、『楽天』=.cache/rakuten-isbn.jsonl から join。

4-1. works(作品)

項目	型	源	用途・備考
slug	TEXT PK	v2	URL・主キー
title / title_kana / title_romaji	TEXT	v2	表示 / 50音ソート・かな検索 / ローマ字検索
subtitle	TEXT	v2	表示(副題)
alt_title_en	TEXT	v2.alternative_title s.en	公式英題=検索・リダイレクト(slugには使わない方針)
alt_titles_json	JSON	v2.alternative_title s	多言語別名
synonyms_json	JSON	v2.synonyms	検索recall。※日本語重複が多く表示には出さない(検索専用)
demographic / status	TEXT	v2	絞り込み(少年/青年… / 連載・完結・休載)
publisher_key / publishers_json	TEXT/JSON	v2	主出版社 / 全出版社(多社作品)
magazine_key	TEXT	v2.magazine	掲載誌で絞り込み
genres_json	JSON	v2.genres	ファセット絞り込み(master32キー)
genres_provisional	BOOL	v2	AI推定ジャンルの低信頼マーク
genres_anilist_json / tags_json	JSON	v2	裏取り/補助 / 要素(あらすじ要素)検索
year_started / year_ended	INT	v2	年代絞り込み・タイムライン
first_volume_date	TEXT	算出	standard版1巻の最小発売日=既定ソートキー
anilist_id	INT	v2	外部リンク・重複検知
wikidata_qid / work_wikidata_qid	TEXT	v2	著者QID / 作品QID(外部リンク)
popularity / score	INT/REAL	v2	人気ソート・ランキング / 評価ソート
adult_us	BOOL	v2	米基準成人=geo出し分け
anime_adapted	BOOL	v2	アニメ化フラグ
catch	TEXT	v2(seed)	キャッチコピー(カード表示)
synopsis_len	INT	算出 len(synopsis)	被覆判定(本文はFTSへ。索引肥大回避で本文は持たない)
volume_count / edition_count	INT	算出	規模
isbn_count / isbn_missing	INT	算出	被覆 / ISBN収獲対象抽出
cover_count / cover_missing / has_cover	INT/BOOL	算出(楽天join)	表紙被覆 / カード表示可否
rep_cover_url	TEXT	算出	代表表紙(1巻優先→表紙ある巻にフォールバック)
search_blob	TEXT	算出	FTSの源(題+かな+ローマ字+英題+別名+著者名を正規化連結)
has_kana/has_romaji/has_synopsis/ has_genres/has_full_isbn/has_anilist/ has_work_qid …	BOOL	算出	被覆フラグ群=監査を1クエリ化。欠けの所在が即わかる

4-2. volumes(巻)

項目	型	源	用途・備考
id / work_slug	INT / TEXT FK	算出	主キー / 親作品
edition_type / edition_label / imprint / edition_publisher	TEXT	v2.editions	版種(通常/完全/愛蔵…)・レーベル・版元
number / is_extra	INT/BOOL	v2	巻番号 / 番外
isbn13 / isbn_present	TEXT/BOOL	v2	ISBN / 有無(ISBN欠落監査)
release_date / date_precision	TEXT/TEXT	v2/算出	発売日 / 精度(day=日まで, month=月まで, none)
cover_url / cover_present / cover_source	TEXT/BOOL/TEXT	楽天join	表紙 / 有無 / 出所(rakuten…)。noimageは無扱い
asin	TEXT	v2	Amazon識別(将来PA-API用)
price / affiliate_url	INT/TEXT	楽天join	価格 / 楽天アフィリエイト(収益導線。最小限のみ保持)

4-3. authors / work_authors(著者)

項目	型	源	用途・備考
authors.key	TEXT PK	算出	著者QIDまたは正規化名
authors.name / kana / romaji	TEXT	v2.authors	表示 / 50音索引・かな検索 / ローマ字検索
authors.alt_names_json	JSON	DB mangaka	表記揺れ(検索recall)
authors.work_count / has_adult_credit	INT/BOOL	算出	著者ページ / 成人signal
work_authors.(work_slug, author_key, role)	—	v2	作品×著者×役割(原作/作画/原作者を分離)

4-4. works_fts(全文検索) / coverage(被覆)

- works_fts = SQLite FTS5。search_blob をトークン化し、題・かな・ローマ字・英題・別名・著者名の部分一致を即時検索。かなは正規化(全角/半角・カタカナ/ひらがな・長音)して取りこぼしを減らす。
- coverage = 各項目の充足件数を持つ集計(ビュー or 物理表)。例:総数・kana有・synopsis有・全巻ISBN有・表紙有 を1行で。年代別・人気帯別の欠けも GROUP BY で即出る。

5. 軽量サイト検索索引(成果物B)の中身

ブラウザが落としても軽いよう1作あたり最小に絞る。かな頭文字でシャード化し、必要シャードだけ遅延ロード。

キー	内容	理由
slug	URL	遷移先
t / k / r	title / かな / ローマ字	3経路で検索ヒット
a	代表著者名	著者でも引ける
p	popularity	候補の並べ替え
d / y	demographic / 年	簡易絞り込み
c	has_cover (0/1)	サムネ表示可否(URLは詳細で取得)

置き場=ビルド出力 out/(R2/Pagesから配信)。gzで~2-3MB、シャード化で初回数+KB。

6. ビルドと更新フロー

- スクリプト scripts/_build-index.py(新規。既存 _build-v2-index.py を発展)。
- 入力 = data/manga.v2/*.yaml(CSafeLoader)+ .cache/rakuten-isbn.jsonl(表紙/価格/アフィリエイト)+ 必要seed。
- 処理 = 正規化(かな/ローマ字)→ search_blob 生成 → 被覆フラグ算出 → SQLite書出(works/volumes/authors/FTS/coverage)→ 軽量JSON生成。
- 更新タイミング = promote(本番再生成)毎・月次蒸留毎。全再構築(冪等)。目安 数分(66k)。

7. 保存・永続化の方針

- マスターSQLite = .cache(再生成可能なので非追跡)。消えても manga.v2 から作り直せる。
- 軽量JSON = ビルド時に out/ へ生成し配信(git追跡しない or 小さいので任意)。
- git追跡するのはレシピ(_build-index.py)だけ=原則2/4に合致。

8. 入れないもの(肥大・陳腐化の回避)

- synopsis本文(全文)→ works表には長さ(被覆)だけ。本文はFTSに入れて検索だけ可能に。
- 楽天の生JSON / 表紙画像そのもの → 持たない(URLとフラグのみ)。
- 種2生層(db-v2)の重複保持 → しない。本索引は公開層専用。
- 頻繁に変わる在庫・リアルタイム価格 → 索引には入れず実行時API(将来)。

9. これで“即答”になる例(イメージ)

- ISBN欠落を年代別に: works/volumes を GROUP BY 年代(今日は数分の走査が必要だった)。
- 表紙が無い人気作 上位: WHERE has_cover=0 ORDER BY popularity DESC。
- あらすじ未充填の連載中作: WHERE has_synopsis=0 AND status='ongoing'。
- かな欠落・slug衝突候補・多社作品 等の品質監査も1クエリ。
- 収穫対象(表紙/ISBN欠け)の抽出も SELECT 一発=次の収穫が即組める。

10. 未決点(着手前に確認したいこと)

- (a) 軽量サイト索引の常設場所: out/配信のみで良いか、検索専用で別ホスト(R2)に置くか。
- (b) FTSのかなトークナイズ: 日本語は分かち書きが必要。N-gram(2-gram)方式にするか(部分一致に強い)。
- (c) 著者キー: QID無し著者の名寄せ(同名異人/表記揺れ)をどこまで索引で吸収するか。
- (d) v2-index.json の扱い: 本索引へ置換(推奨)か、当面併存か。
- (e) 成人作の索引: adult_usは持つが、軽量索引(配信)にどう含める/隠すか(geoはWorker側)。

以上。承認 or 修正点をいただければ、この仕様で `_build-index.py` を実装します。